Retrieval-Augmented Generation (RAG) Platform

Domain	Difficulty	Team Size	Timeline	Primary Stack
NLP · Enterprise AI	Intermediate	2 Members	12 Weeks	Python · Next.js · PostgreSQL

1. THE PROBLEM

Every organisation — a university library, a government department, a law firm, or a small business — accumulates hundreds of documents over time. These documents exist in multiple languages (English, Hindi, Marathi, and more in India's context), and they are completely unsearchable beyond keyword matching. When someone needs an answer, they manually read through files, waste hours, and often miss critical information buried deep inside.

Traditional search fails at meaning	Keyword search can find the word 'deadline' but cannot answer 'when does the policy expire?' — it doesn't understand intent or context.
Multilingual documents are invisible to most tools	A Hindi circular alongside an English policy cannot be queried together. Existing tools are either English-only or require separate pipelines per language.
LLMs hallucinate without grounding	Directly asking a general-purpose LLM (ChatGPT, Gemini) about private documents produces plausible-sounding but fabricated answers — there is no citation, no source, no trust.
No citation trail	Decision-makers and researchers need to know exactly which paragraph in which document produced an answer, so they can verify it. Existing chat AI provides none.

2. WHAT WE ARE BUILDING

A production-grade Multilingual Document Intelligence SaaS — a web platform where users upload documents in any language, and can then ask questions in natural language (in any language) and receive precise, cited, grounded answers drawn exclusively from their own documents.

■ Core User Journey

Step 1 Upload	User uploads PDFs, Word docs, or plain text — in any language — through a drag-and-drop web interface.
Step 2 Process	The system extracts text, detects the language, splits into semantic chunks, and generates multilingual embeddings using the multilingual-E5 model.
Step 3 Store	Chunk embeddings + metadata are stored in pgvector (a PostgreSQL extension), indexed for sub-second similarity search across all documents.
Step 4 Query	User types a question in any language. The query is embedded, and the top-K most semantically relevant chunks are retrieved via cosine similarity.
Step 5 Generate	The retrieved chunks + user question are passed as context to Claude API. Claude generates a grounded, precise answer — it cannot invent facts outside the provided context.
Step 6 Cite	Every answer is displayed alongside the exact source paragraph, document name, page number, and language — full traceability.

■ Key Features

- Multi-tenant architecture — each organisation's documents are isolated and private
- Cross-lingual queries — ask in English, retrieve from Hindi/Marathi documents
- Source citations with page-level granularity on every answer
- Document library with upload history, language tags, and status indicators
- Conversation memory — follow-up questions stay in context within a session
- Usage analytics dashboard — query frequency, top documents, response latency
- JWT-secured auth + per-user rate limiting
- Configurable chunk size and retrieval top-K parameters

3. HOW IT SOLVES THE PROBLEM

RAG (Retrieval-Augmented Generation) is the architectural pattern that makes this possible. It combines the precision of semantic vector search with the fluency of a large language model — but crucially, the LLM is constrained to speak only about what is in the retrieved documents. This eliminates hallucination and produces auditable, trustworthy answers.

Problem Traditional keyword search misses semantic meaning	→	**Solution** multilingual-E5 embeddings encode semantic meaning — 'treaty expiry' and 'agreement end date' are recognised as the same concept
Problem Multilingual documents are invisible to English-only tools	→	**Solution** multilingual-E5 supports 100+ languages in a single embedding space — a Hindi question returns the right English paragraph and vice versa
Problem LLMs hallucinate without grounding	→	**Solution** Claude is given only the retrieved chunks as context. Its system prompt explicitly forbids answering outside the provided text, eliminating fabrication
Problem No citation trail	→	**Solution** Every chunk in the vector store carries metadata: document name, page, language, position. Citations are rendered alongside every answer automatically

Novel Value

Most RAG tutorials use single-language English documents. This project's novel contribution is the multilingual embedding layer (multilingual-E5) that allows cross-lingual retrieval — a feature absent from most existing open-source RAG stacks and directly relevant to India's multilingual administrative and academic context.

4. TECHNICAL ARCHITECTURE

Frontend Layer	*Next.js 14 (App Router) · TypeScript · Tailwind CSS · shadcn/ui*	Document upload UI, chat Q&A; interface, citation viewer, user dashboard, organisation settings. Deployed to Vercel.
API Gateway	*Next.js API Routes · JWT Middleware · Rate Limiting*	Validates auth tokens, enforces per-user rate limits, routes requests to the appropriate FastAPI microservice.

Ingestion Service	*FastAPI · pdfplumber · python-docx · LangChain Text Splitter · langdetect*	Accepts uploaded files, extracts raw text (PDF/DOCX/TXT), detects language, splits into 512-token overlapping chunks, passes chunks to the embedding service.
Embedding Service	*HuggingFace multilingual-E5-large · sentence-transformers · PyTorch*	Converts each text chunk into a 1024-dimensional dense vector. Supports 100+ languages in a single unified embedding space — no separate models per language.
Vector Store	*PostgreSQL + pgvector extension · SQLAlchemy ORM*	Stores chunk text, embedding vector, and metadata (doc_id, page, language, chunk_index). Cosine similarity search via pgvector's <=> operator. Indexed with HNSW for sub-50 ms retrieval on large corpora.
RAG Orchestration	*LangChain · Claude API (claude-haiku-3-5) · Prompt Engineering*	Retrieves top-5 chunks for a query, assembles a grounded system prompt, calls Claude API with retrieved context, streams the response token by token to the frontend via Server-Sent Events (SSE).
Storage & Auth	*AWS S3 (raw files) · PostgreSQL (users, orgs, docs) · JWT*	Raw document files stored in S3 with pre-signed URLs. Multi-tenant user schema: organisations → users → document_collections → chunks.

5. HOW IT WORKS — DATA FLOW

■ A. Document Ingestion Pipeline

1	User uploads file via drag-and-drop UI (PDF / DOCX / TXT)
2	Frontend POSTs the file to /api/ingest — JWT validates the request
3	FastAPI saves raw file to AWS S3, returns a file_key
4	pdfplumber (PDF) or python-docx (DOCX) extracts plain text per page
5	langdetect identifies the language of each page (e.g. en, hi, mr)
6	LangChain RecursiveCharacterTextSplitter chunks text into 512-token windows with 64-token overlap
7	multilingual-E5-large encodes each chunk to a 1024-dim float32 vector
8	Each (chunk_text, embedding, metadata) row is inserted into PostgreSQL + pgvector
9	Document status updated to 'ready'; user notified via Server-Sent Event

■ B. Query & Generation Pipeline

1	User types a question in the chat interface (any language)
2	Query text is embedded with multilingual-E5-large → 1024-dim vector
3	pgvector executes: SELECT chunk_text, metadata, 1-(embedding <=> query_vec) AS score FROM chunks WHERE org_id=$1 ORDER BY score DESC LIMIT 5
4	Top-5 chunks (with source metadata) are assembled into a context block
5	A grounded system prompt is constructed: 'Answer ONLY from the context below. If the answer is not in the context, say so.'
6	Claude API (claude-haiku-3-5) is called with the context + user question
7	Claude's response is streamed via SSE to the frontend as it generates
8	After the stream ends, citations (doc name, page, language, snippet) are appended below the answer
9	The conversation turn (query + answer + citations) is saved to the sessions table for follow-up queries

6. TEAM DIVISION — PARALLEL WORKSTREAMS

The project is split into two parallel tracks that can be developed independently and integrated at defined sync points. Both members work simultaneously from Day 1 with clear interface contracts — the API schema is agreed upfront so the frontend can be built with mock data while the backend is being implemented.

Member 1 — AI / ML · Backend · DevOps

AI / ML Pipeline

Backend (FastAPI)

Vector Database

RAG Orchestration

Docker & Deployment

- Set up FastAPI project, Docker, environment config
- Implement document text extraction (pdfplumber, python-docx)
- Integrate langdetect for per-page language tagging
- Implement LangChain text chunking (recursive + semantic)
- Set up multilingual-E5-large embedding pipeline (HuggingFace)
- Configure PostgreSQL + pgvector, write schema migrations
- Build cosine similarity search queries (HNSW index)
- Implement RAG orchestration: retrieval → prompt assembly → Claude API call
- Add SSE streaming for real-time answer delivery
- Write /ingest, /query, /collections FastAPI endpoints
- Containerise with Docker Compose (FastAPI + PostgreSQL)
- Deploy backend to Railway / Render with environment secrets

Member 2 — Frontend · Auth · Schema · QA

Frontend (Next.js)

Auth & Multi-tenancy

Database Schema

UI/UX Design

Testing & QA

- Set up Next.js 14 project (App Router, TypeScript, Tailwind, shadcn/ui)
- Implement JWT auth flow + protected routes middleware
- Build multi-tenant schema: organisations, users, documents tables
- Create document upload page (drag-and-drop, progress bar, language tag display)
- Build chat Q&A; interface with SSE streaming message renderer
- Build citation card component (doc name, page, highlighted snippet)
- Create document library page (list, status, delete, search)
- Build user dashboard with usage stats (Recharts charts)
- Integrate all FastAPI endpoints (with Axios/fetch + React Query)
- Add error states, loading skeletons, empty states throughout
- Deploy frontend to Vercel with environment variable management
- Write end-to-end tests for upload + query flows

■ Integration Sync Points

Week 2 end	API contract agreed (request/response JSON schemas) — frontend uses mock data after this

Week 5 end	Backend /ingest endpoint live — frontend switches from mock to real upload calls
Week 7 end	Backend /query + SSE endpoint live — frontend switches chat to live calls
Week 9 end	Full integration complete — begin joint testing with real multilingual documents
Week 11 end	Demo-ready build — final polishing and documentation

7. 12-WEEK PROJECT TIMELINE

Both members work in parallel throughout. Shading indicates which member owns each phase.

Phase 1	Foundation & Setup	Weeks 1–2	• Agree on API contract (JSON schemas for /ingest, /query) • M1: FastAPI skeleton, Docker Compose, PostgreSQL + pgvector setup • M2: Next.js project, Tailwind/shadcn config, JWT auth pages • Both: Set up GitHub repo with branch strategy and CI	**Both**
Phase 2	Core ML Pipeline	Weeks 3–5	• Text extraction (pdfplumber, python-docx) — PDF, DOCX, TXT support • Language detection with langdetect • LangChain chunking (512 tokens, 64 overlap) • multilingual-E5-large embedding pipeline • pgvector HNSW index, /ingest endpoint live	**Member 1**
Phase 3	Frontend Shell	Weeks 3–5	• Document upload page with drag-and-drop and progress UI • Document library page (mock data) • Chat Q&A; interface with streaming message renderer (mock SSE) • Citation card component • User dashboard with Recharts charts (mock data)	**Member 2**
Phase 4	RAG Orchestration	Weeks 6–7	• pgvector cosine similarity search query • Context assembly + grounded Claude system prompt • Claude API integration with SSE streaming • /query endpoint live • Conversation session persistence	**Member 1**
Phase 5	Integration & Auth	Weeks 6–7	• Multi-tenant PostgreSQL schema (orgs, users, collections) • Switch frontend from mock to live /ingest calls • Switch chat from mock SSE to live /query stream • JWT middleware, protected routes, rate limiting UI	**Member 2**

Phase 6	Full Integration	Weeks 8–9	• End-to-end testing with real multilingual documents (Hindi + English PDFs) • M1: performance tuning (batch embedding, connection pooling) • M2: error states, loading skeletons, mobile responsiveness • Joint: cross-lingual query accuracy benchmarking	Both
Phase 7	Deployment & Polish	Weeks 10–11	• M1: Deploy FastAPI to Railway/Render, configure secrets • M2: Deploy Next.js to Vercel, configure environment variables • Both: smoke test production, fix production-specific issues • Both: write API documentation (Swagger auto-gen)	Both
Phase 8	Demo & Documentation	Week 12	• Record demo video with multilingual query examples • Write project report (problem, architecture, results, future work) • Prepare presentation slides • Final code cleanup, README, and open-source release	Both

8. COMPLETE TECHNOLOGY STACK

Category	Technology	Purpose
Language	Python 3.11, TypeScript	Backend ML/API + Frontend
Frontend	Next.js 14, Tailwind CSS, shadcn/ui	SaaS web application UI
Backend	FastAPI, Uvicorn	Document ingestion + RAG query API
Embedding Model	multilingual-E5-large (HuggingFace)	100+ language unified embeddings
Vector DB	PostgreSQL 16 + pgvector	Semantic similarity search store
ORM / DB	SQLAlchemy, Alembic migrations	Database access and schema versioning
Document Parsing	pdfplumber, python-docx, langdetect	Text extraction + language detection
Text Chunking	LangChain RecursiveCharacterTextSplitter	512-token overlapping windows
LLM (Generation)	Anthropic Claude API (claude-haiku-3-5)	Grounded answer generation
RAG Framework	LangChain (partial)	Pipeline orchestration
Streaming	Server-Sent Events (SSE)	Real-time token-by-token answer streaming
File Storage	AWS S3	Raw document file storage

Category	Technology	Purpose
Auth	JWT, bcrypt	Stateless multi-tenant authentication
Caching	Redis (Upstash)	Query result caching, session state
Containerisation	Docker, Docker Compose	Reproducible dev and prod environments
Frontend Deploy	Vercel	Next.js hosting with CI/CD
Backend Deploy	Railway or Render	FastAPI + PostgreSQL hosting
Testing	Pytest (backend), Vitest + Testing Library (frontend)	Unit and integration tests
Monitoring	Sentry	Error tracking in production

9. DATASET, EVALUATION & NOVELTY

■ Test Documents (Free & Publicly Available)

AI4Bharat IndicCorp	Hindi, Marathi, Telugu + 11 other Indian languages — large text corpora
NPTEL PDFs	English engineering lectures — high-quality technical PDF content
Government of India Official Gazettes	Mix of English and Hindi — real multilingual policy documents
Wikipedia Dumps	Any language — good for cross-lingual Q&A; benchmarking
Custom test set	15–20 documents assembled by the team across English + Hindi + Marathi

■ Evaluation Metrics

- Retrieval Precision@5 — are the top-5 retrieved chunks actually relevant to the query?
- Answer Faithfulness — does the generated answer only use information from retrieved chunks? (manual check + LLM-as-judge)
- Cross-lingual hit rate — percentage of Hindi queries that correctly retrieve English chunks and vice versa
- Mean response latency — end-to-end time from query to first token (target: under 2 seconds)
- User study — 5–10 test users rate answer quality and citation accuracy on a 1–5 scale

Novel Contribution	Cross-lingual RAG for Indian languages (Hindi, Marathi, English) in a single multi-tenant SaaS — directly applicable to government, legal, and educational use-cases in India. This goes beyond standard English-only RAG tutorials and adds real-world deployment complexity (multi-tenancy, SSE streaming, production auth).

10. THE REAL ML CORE — BEYOND API CALLS

The problem	Calling Claude or Gemini API is an HTTP request — not ML work. For a final year project, you need hands-on model training, fine-tuning, or significant ML engineering that you own end-to-end. This section defines the three ML components that make this project academically defensible.

■ ML Track 1 — Fine-Tune the Embedding Model (Core, Recommended)

Instead of using multilingual-E5-large as a frozen off-the-shelf model, you fine-tune it on a custom retrieval dataset using contrastive learning. The model learns to pull semantically related query-chunk pairs closer in vector space and push unrelated pairs apart. This directly improves retrieval precision — the most critical metric in RAG.

Dataset Construction	From your document collection, generate (query, positive_chunk, negative_chunk) triplets. Positives: chunks that answer the query. Negatives: chunks from the same document that are off-topic. Use GPT-3.5/Gemini to auto-generate 2,000–5,000 query pairs cheaply. Annotate a subset manually for evaluation.
Training Objective	Multiple Negatives Ranking Loss (InfoNCE) — for each query embedding, maximise cosine similarity to the positive chunk and minimise it to all negatives in the batch. This is implemented in the sentence-transformers library in ~30 lines of Python.
Training Setup	Use Google Colab Pro (A100 GPU, free tier works too). Fine-tune for 3–5 epochs with a batch size of 16–32, learning rate 2e-5, warmup steps 10%. Total training time: 2–4 hours on a T4 GPU. Save the checkpoint to HuggingFace Hub.
Evaluation	Measure Precision@5 and NDCG@10 on a held-out evaluation set before and after fine-tuning. A well-tuned model typically improves P@5 by 8–15% on domain-specific queries versus the generic off-the-shelf model.
Integration	Replace the generic multilingual-E5 in the ingestion and query pipelines with your fine-tuned checkpoint. Load via sentence_transformers.SentenceTransformer('your-model'). Re-embed all existing documents with the new model.

■ ML Track 2 — Train a Cross-Encoder Re-Ranker (Second ML Model)

The bi-encoder embedding model (Track 1) is fast but approximate — it compresses each text into a single vector, losing fine-grained interaction between query and chunk. A cross-encoder re-ranker reads the query and chunk together and scores their relevance with full attention. You retrieve top-20 chunks with pgvector, re-rank with this model, and pass only the top-5 to the LLM — dramatically improving answer quality.

Architecture	Fine-tune a small BERT-base-multilingual-cased model as a binary classifier: input is [CLS] query [SEP] chunk [SEP], output is a relevance score 0–1. This is a sequence-pair classification task using HuggingFace AutoModelForSequenceClassification.
Dataset	Re-use the same triplets from Track 1. Label positive pairs as 1, negative pairs as 0. Add hard negatives (chunks that are topically similar but don't answer the query) for a challenging training signal. 3,000–8,000 pairs are sufficient.
Training	Binary cross-entropy loss, AdamW optimizer, 3 epochs, batch size 32. Runs in under 1 hour on a T4 GPU. Much lighter than fine-tuning the embedding model.
Integration	After pgvector returns top-20 candidates, score each (query, chunk) pair with the re-ranker, sort by score descending, take top-5. Adds ~100–200 ms latency but significantly improves final answer relevance.

■ ML Track 3 — Fine-Tune a Local LLM with QLoRA (Replace API)

The most ambitious option: replace Claude API entirely with a locally fine-tuned small language model. Using QLoRA (Quantized Low-Rank Adaptation), you can fine-tune a 3B–7B parameter model on a consumer GPU (Google Colab A100, 16 GB VRAM) in a few hours. The model is trained to answer questions strictly from provided context — eliminating hallucination by design.

Aspect	Detail
Base model	Llama-3.2-3B-Instruct or Mistral-7B-Instruct (both Apache 2.0 licensed)
Fine-tuning method	QLoRA — 4-bit quantisation + LoRA adapters. Only ~1% of parameters are trained.
Training data	Instruction-tuning dataset: (context, question, grounded_answer) triples. Generate 1,000–2,000 examples from y
Training infra	Google Colab A100 (free tier). Uses bitsandbytes, peft, trl libraries. Training time: 2–5 hours depending on datas
Deployment	Export to GGUF format, run via llama.cpp on CPU (no GPU needed in prod). Serve via llama-cpp-python FastAP
Tradeoff vs API	Lower quality than Claude/GPT-4 but fully yours — no API cost, no data leaves your server, stronger ML project

■ Recommended Approach for This Project

Track 1: Fine-tune multilingual-E5 on domain triplets using InfoNCE loss (sentence-transformers)	Member 1	Weeks 4–6
Track 2: Train BERT cross-encoder re-ranker for top-K re-ranking (adds second ML model)	Member 1	Weeks 6–7

Keep Claude API for generation (or swap with QLoRA LLM in Track 3 if time allows)	Member 1	Weeks 3+
Benchmark: compare generic E5 vs fine-tuned E5 vs E5 + re-ranker on P@5 and NDCG@10	Both	Week 8

Revised ML Statement

"We fine-tuned a multilingual sentence embedding model using contrastive learning (InfoNCE loss) on a custom trilingual retrieval dataset, and trained a BERT-based cross-encoder re-ranker to improve retrieval precision. Together these reduce the generation layer's hallucination surface and improve Precision@5 by ~12% over the generic baseline. The generation step uses a locally hosted QLoRA-fine-tuned LLM (or Claude API as a fallback) grounded strictly to retrieved context."

11. FUTURE ENHANCEMENTS

The following enhancements are scoped beyond the 12-week final year project. They are organised into four tracks: AI/ML upgrades, product features, enterprise capabilities, and performance engineering. These can be presented as a future roadmap in your project report and demo, and can be built post-submission to grow the SaaS.

■ Track A — AI / ML Upgrades

GraphRAG — Knowledge Graph Layer	Extract entities and relationships from documents using spaCy NER + a relation extraction model, build a knowledge graph (Neo4j or NetworkX), and enable graph-traversal queries ('What are all the policies related to entity X?'). Dramatically improves multi-hop reasoning that flat vector search cannot handle.
Multimodal RAG — Tables, Charts & Images	Most PDFs contain tables and embedded images that pdfplumber cannot parse meaningfully. Integrate a vision model (PaddleOCR for tables, a CLIP-based model for images) to extract structured content from visual elements and make them queryable alongside text.
Agentic RAG — Multi-Step Reasoning	Instead of a single retrieve-then-generate call, build an agent loop (LangChain AgentExecutor or LlamaIndex ReAct agent) that can decompose complex queries into sub-questions, retrieve for each, and synthesise a multi-step answer with intermediate reasoning shown to the user.
Adaptive Chunking with Semantic Boundaries	Replace fixed-token chunking with a model that detects semantic topic boundaries (e.g. fine-tuned sentence-BERT topic segmentation). Chunks that respect paragraph/section semantics produce cleaner retrieval than arbitrary token windows.
Hallucination Scoring / Faithfulness Checker	Train a small NLI (natural language inference) model to score whether each sentence in the generated answer is entailed by the retrieved context. Flag low-confidence sentences in the UI with a warning colour before the user acts on them.
Continuous Learning from User Feedback	Add thumbs up/down feedback on answers. Use negative feedback to generate hard negative training examples for the embedding model. Re-fine-tune the embedder monthly — the system gets better the more it is used.

■ Track B — Product / SaaS Features

Voice Input and Audio Answers	Integrate Whisper (speech-to-text) for voice queries — users speak their question, the system queries the docs, and a text-to-speech model (gTTS or ElevenLabs API) reads the answer back. Makes the platform accessible on mobile without typing.

WhatsApp / Telegram Bot Integration	Expose the /query API via a WhatsApp Business API or Telegram bot. Users can send a message to a number and receive grounded document answers in chat — zero UI friction for non-technical users in government, legal, or NGO settings.
Document Comparison Mode	Allow users to select two or more documents and ask comparison questions ('How does Policy A differ from Policy B on leave entitlement?'). Implement with a two-branch retrieval that fetches chunks from each document separately, then passes them side-by-side to the LLM for structured comparison output.
Collaborative Annotations	Real-time multi-user document annotation layer (similar to Notion or Google Docs comments). Annotations are themselves indexed in pgvector, so team knowledge ('this clause is disputed per legal review') becomes queryable alongside the source text.
Google Drive / Dropbox / SharePoint Sync	OAuth-based integration that watches a cloud folder for new or updated documents and automatically re-ingests and re-indexes them. Organisations no longer need to manually re-upload — the knowledge base stays current automatically.
Browser Extension for Web Pages	A Chrome/Firefox extension that lets users ingest any web page into their document collection with one click. Useful for researchers who want to query across saved articles alongside their own documents.
Scheduled Summarisation Reports	Weekly email digest that summarises the most-queried topics, highlights newly added documents, and surfaces low-confidence answers flagged by the faithfulness scorer. Built with a cron job + SendGrid.

■ Track C — Enterprise Capabilities

On-Premises / Air-Gapped Deployment	Package the entire stack (FastAPI + PostgreSQL + pgvector + local LLM via llama.cpp) into a single Docker Compose bundle that runs on an organisation's own server with no internet dependency. Critical for government, defence, healthcare, and legal use-cases where data cannot leave the premises.
Role-Based Access Control (RBAC)	Document-level permissions: owner, editor, viewer, no-access roles per document collection. A legal associate can query the contracts folder but not the HR policies folder. Implemented as row-level security in PostgreSQL.
Full Audit Trail and Compliance Logging	Every query, document upload, user action, and answer generated is logged with timestamp, user ID, and IP address. Export audit logs as PDF/CSV for compliance reviews (GDPR, DPDP Act India). Immutable log store with append-only PostgreSQL table.

Developer API (REST + Webhooks)	Expose a public REST API with API key authentication so organisations can integrate the platform into their own internal tools, dashboards, or automation pipelines without using the web UI. Add webhook events for document-processed and query-answered.
Single Sign-On (SSO) — Google Workspace & Microsoft Azure AD	Enterprise authentication via SAML 2.0 / OIDC. Organisation employees log in with their existing corporate credentials. Removes the need for separate password management and supports Just-in-Time provisioning for large teams.
Usage Metering and Billing	Per-organisation query quotas, document storage limits, and tiered pricing plans (Free / Pro / Enterprise). Integrate Stripe for subscription billing, with a real-time usage dashboard showing queries consumed vs. plan limit.

■ Track D — Performance Engineering

Hybrid Search — BM25 + Dense Vector	Combine BM25 keyword retrieval (PostgreSQL full-text search tsvector) with pgvector cosine similarity using Reciprocal Rank Fusion (RRF). Hybrid search consistently outperforms pure vector search on factual queries with specific named entities, dates, or numbers that embeddings tend to blur.
Semantic Caching of Query Results	Before hitting the vector store, check Redis for a semantically similar cached query (cosine similarity > 0.95 between the new query embedding and cached query embeddings). Return the cached answer instantly. Reduces LLM API calls by 20–40% for common queries.
Asynchronous Batch Embedding	For large document uploads (100+ pages), queue embedding jobs in a Celery + Redis worker pool rather than blocking the HTTP request. Users see a real-time progress bar and are notified when the document is ready. Already familiar from your Pehno pipeline.
HNSW Index Tuning and Quantisation	Tune pgvector HNSW parameters (ef_construction, m) for the specific corpus size. Apply scalar quantisation (SQ8) to reduce embedding storage by 4x and improve cache locality. At 1M+ chunks, the difference becomes significant.
Edge Deployment for Embedding	Compile multilingual-E5 to ONNX and quantise with ONNX Runtime. The quantised model runs 3–5x faster than the PyTorch version with negligible quality loss, making sub-50 ms embedding generation feasible on CPU-only servers.

■ Enhancement Priority Matrix

Enhancement	Track	Effort	Impact	When
Hybrid Search (BM25 + Vector)	Performance	Low	High	Post v1.0
Semantic Query Caching	Performance	Low	High	Post v1.0
Voice Input (Whisper)	Product	Low	High	v1.1
Hallucination Scorer	AI/ML	Medium	High	v1.1
Document Comparison Mode	Product	Medium	High	v1.1
Telegram / WhatsApp Bot	Product	Medium	Medium	v1.2
Google Drive Sync	Product	Medium	High	v1.2
RBAC — Document-Level Permissions	Enterprise	Medium	High	v2.0
Audit Trail + Compliance Logs	Enterprise	Low	High	v2.0
Developer REST API + Webhooks	Enterprise	Medium	High	v2.0
GraphRAG (Knowledge Graph)	AI/ML	High	High	v2.1
Agentic Multi-Step RAG	AI/ML	High	High	v2.1
Multimodal RAG (Tables + Images)	AI/ML	High	High	v2.1
On-Premises Docker Bundle	Enterprise	High	High	v2.2
Continuous Learning from Feedback	AI/ML	High	Medium	v2.2
SSO — Google / Azure AD	Enterprise	High	Medium	v3.0
Usage Metering + Stripe Billing	Enterprise	Medium	High	v3.0
Edge ONNX Embedding	Performance	Medium	Medium	v3.0